التقرير التوثيقي المعماري والأمني الشامل

عزل المستأجرين (Tenant Isolation) والتفويض الدقيق (Fine-Grained Authorization) — Milestones 4 & 5

المشروع:	**Property Management SaaS (.NET 10)**	حالة الاختبارات:	**Passed (92/92 100% Tests)**	حالة الـ CI:	**Green Success** ✅
التاريخ:	**نوفمبر 2026**				

1. المقدمة ونطاق العمل (Executive Overview)

في أنظمة إدارة العقارات متعددة المستأجرين (Multi-Tenant SaaS)، يعد أمان البيانات والفصل الصارم بين الشركات (Organizations) هو حجر الزاوية للمنظومة بأكملها. لا يمكن لنظام إنتاجي أن يسمح تحت أي ظرف لشركة بالاطلاع على عقارات أو وحدات أو مستأجري شركة أخرى، أو تعديلها.

ركزت هذه المرحلة على تحقيق هدفين رئيسيين:

1. **عزل المستأجرين التام (Fail-Closed Multi-Tenancy):** ضمان أن أي عملية استعلام أو تعديل أو إضافة تكون محصورة تلقائيًا داخل سياق الشركة المعتمدة، مع الرفض الفوري والحاسم لأي طلب بدون سياق أو طلب عابر للمستأجرين (Cross-Tenant).
2. **نظام التفويض المبني على الصلاحيات (Permission-Based Authorization):** الانتقال من مجرد الرتب العامة (RBAC) إلى صلاحيات دقيقة (Fine-Grained Permissions) تتيح التحكم الكامل بكل Endpoint استناداً لدور المستخدم داخل المنظمة.

2. ما تم إنجازه تفصيلياً مع شرح الأكواد سطرًا بسطر

أولاً: استخلاص سياق المستخدم الحالي (ICurrentUser & CurrentUser)

تم إنشاء تجريد `ICurrentUser` في طبقة الـ Application لتمكين جميع Use Cases و DbContext من معرفة هوية المستخدم الحالي والشركة التي ينتمي إليها دون الاعتماد المباشر على `HttpContext`:

```
// CurrentUser.cs
public sealed class CurrentUser(IHttpContextAccessor httpContextAccessor) : ICurrentUser
{
    private ClaimsPrincipal? User ⇒ httpContextAccessor.HttpContext?.User;
    public bool IsAuthenticated ⇒ User?.Identity?.IsAuthenticated ?? false;

    public Guid? OrganizationId
    {
        get
        {
            var claimValue = User?.FindFirstValue("organization_id")
                ?? User?.FindFirstValue("tenant_id");
            return Guid.TryParse(claimValue, out var id) ? id : null;
        }
    }
    // ... UserId, Role, Email
}
```

شرح الكود: نقرأ الـ `ClaimsPrincipal` من الـ JWT المعتمد. نستخرج الـ `organization_id` المشفر والموقع رقميًا في الـ Token. هذا يمنع أي عميل من تزييف الـ ID لأن الـ Token موقع بمفتاح السيرفر السري (HMAC-SHA256).

ثانياً: عزل البيانات الصارم (Fail-Closed Isolation) في ApplicationDbContext

تم تطبيق فلاتر استعلام عامة (Global Query Filters) على جميع الكيانات التابعة للمنظمات (`Tenant` , `Unit` , `Property`):

```
// ApplicationDbContext.cs - OnModelCreating
builder.Entity<Property>().HasQueryFilter(p ⇒
    p.OrganizationId == (_currentUser ≠ null && _currentUser.IsAuthenticated && _currentUser.
        ? _currentUser.OrganizationId.Value
        : Guid.Empty));
```

سر الحماية الفائقة (Fail-Closed Architecture):

إذا كان المستخدم غير مسجل، أو مسجل بدون منظمة، أو إذا كان الـ Request قادمًا من خلفية غير محددة، الشرط يتحول إلى: `p.OrganizationId == Guid.Empty` . وبما أنه لا يوجد أي عقار في قاعدة البيانات يحمل معرف `Guid.Empty` ، فإن قاعدة البيانات ترجع **صفر نتائج فوراً**، ويستحيل تسريب أي سجل.

ثالثاً: حماية عمليات الكتابة (Write-Side Isolation) في SaveChangesAsync

الفلاتر وحدها تحمي القراءة، لكن ماذا لو حاول مستخدم خبيث إرسال Entity جديدة بـ `OrganizationId` يخص شركة أخرى؟ تم سد هذه الثغرة عبر الـ Interception في `SaveChangesAsync` :

```
// ApplicationDbContext.cs - SaveChangesAsync
if (_currentUser?.IsAuthenticated == true && _currentUser.OrganizationId.HasValue)
{
    var currentOrgId = _currentUser.OrganizationId.Value;

    foreach (var entry in ChangeTracker.Entries<IOrganizationScoped>())
    {
        if (entry.State == EntityState.Added)
        {
            // ختم المعرف تلقائيًا لو كان فارغاً
            if (entry.Entity.OrganizationId == Guid.Empty)
                entry.Property(nameof(IOrganizationScoped.OrganizationId)).CurrentValue = curr
            // رفض قاطع لو كان المعرف لا يطابق المنظمة المعتمدة
            else if (entry.Entity.OrganizationId ≠ currentOrgId)
                throw new InvalidOperationException("Cross-tenant write operation detected.");
        }
        else if (entry.State == EntityState.Modified)
        {
            if (entry.Entity.OrganizationId ≠ currentOrgId)
                throw new InvalidOperationException("Cross-tenant modification detected.");
        }
    }
}
```

رابعاً: نظام التفويض الديناميكي الدقيق (Fine-Grained PBAC)

بدلاً من كتابة `[Authorize(Roles = "Owner,Admin")]` على كل Controller، أنشأنا نظام تصاريح مرن:

1. **كتالوج الصلاحيات الثابت (Permissions Catalog):**

```
public static class Permissions
{
    public static class Properties {
        public const string Read = "Permissions.Properties.Read";
        public const string Create = "Permissions.Properties.Create";
        public const string Update = "Permissions.Properties.Update";
        public const string Archive = "Permissions.Properties.Archive";
    }
    // Units, Tenants, Billing...
}
```

2. **الـ Custom Attribute** `[HasPermission( ... )]` :

```
[AttributeUsage(AttributeTargets.Method | AttributeTargets.Class, AllowMultiple = true)]
public sealed class HasPermissionAttribute(string permission) : AuthorizeAttribute(policy:
```

3. **الـ Dynamic Policy Provider:** يقوم بإنشاء Authorization Policy في الذاكرة لأي صلاحية تبدأ بـ `Permissions.` دون الحاجة لتعريف مئات الـ Policies يدويًا في `Program.cs` :

```
if (policyName.StartsWith("Permissions.", StringComparison.OrdinalIgnoreCase))
{
    return new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .AddRequirements(new PermissionRequirement(policyName))
        .Build();
}
```

4. **الـ Authorization Handler ومصفوفة الصلاحيات:** يفحص الـ Handler دور المستخدم (`MembershipRole`) داخل المنظمة ويتحقق مما إذا كان هذا الدور يمتلك الصلاحية المطلوبة:

الصلاحية	Owner / Admin	PropertyManager	Staff
Properties.Read / Units.Read / Tenants.Read	مسموح	مسموح	مسموح
Properties.Create / Update / Units.Create / Update	مسموح	مسموح	ممنوع
Properties.Archive / Tenants.Blacklist	مسموح	ممنوع	ممنوع

خامساً: تأمين الـ Controllers وإغلاق ثغرة الـ IDOR

تم تنظيف `PropertiesController` و `UnitsController` بالكامل:

```
// PropertiesController.cs
[Authorize]
[ApiController]
[Route("api/v1/[controller]")]
public sealed class PropertiesController(ISender sender, ICurrentUser currentUser) : Controlle
{
    private Guid? CurrentOrgId ⇒ currentUser.OrganizationId;

    [HttpGet("{id:guid}")]
    [HasPermission(Permissions.Properties.Read)]
    public async Task<IActionResult> GetById(Guid id, CancellationToken ct)
    {
        if (!CurrentOrgId.HasValue)
            return Unauthorized(new ProblemDetails { Detail = "An active organization context

        var query = new GetPropertyByIdQuery(CurrentOrgId.Value, id);
        var result = await sender.Send(query, ct);
        return result.IsFailure ? this.ToActionResult(result) : Ok(result.Value);
    }
}
```

3. الأخطاء والثغرات التي واجهتنا أثناء التطوير وكيف تم حلها

1. الخطأ الكارثي: العزل بطريقة Fail-Open (Data Leakage Vulnerability)

وصف المشكلة: كان الفلتر المكتوب في البداية يحتوي على الشرط التالي:

```
builder.Entity<Property>().HasQueryFilter(p ⇒
    _currentUser == null ||
    !_currentUser.IsAuthenticated ||
    !_currentUser.OrganizationId.HasValue ||
    p.OrganizationId == _currentUser.OrganizationId.Value);
```

الأثر الكارثي: إذا كان الـ Request قادمًا من مستخدم غير مصادق أو JWT بدون claim، فإن الشروط الثلاثة الأولى تُرجع `true`. هذا كان يجعل EF Core يُلغي الفلتر تمامًا وينفذ `SELECT * FROM Properties` ليعرض بيانات كل الشركات!

الحل الجذري: تحويل المعادلة إلى Fail-Closed؛ حيث يتم فحص وجود المنظمة، فإذا غابت يُقارن المعرف بـ `Guid.Empty`، مما يؤدي دائمًا إلى إرجاع صفر سجلات.

2. ثغرة IDOR عبر تجاوز الـ Query String في الـ Controllers

وصف المشكلة: كان في الـ Endpoints كود بالشكل التالي:

```
var targetOrgId = CurrentOrgId ?? organizationId; // حيث organizationId قادم من [FromQuery
```

الأثر: لو كان المستخدم مصادقًا ولكن ليس لديه claim Organization، يستطيع تمرير `?organizationId=ANY_COMPANY_GUID` في الرابط والوصول لبياناتها!

الحل الجذري: مسح المعامل `[FromQuery] Guid? organizationId` نهائيًا من كل الـ Endpoints والاعتماد حصرًا على التوكن الموثق، مع رد حاسم بـ `401 Unauthorized` لو لم يتوفر Context.

3. خطأ تضارب الأسماء في Swagger (.NET 10 Build Failure)

وصف المشكلة: ظهر خطأ كومبايلر أثناء محاولة إضافة Swagger Security Definition:

```
error CS0234: The type or namespace name 'Models' does not exist in the namespace 'Micros
```

السبب والحل: في .NET 10، هناك تحديث في مكتبات `Microsoft.OpenApi`. قمنا بإعادة استخدام التهيئة القياسية لـ Swashbuckle المتوافقة مع .NET 10، وتم حل المشكلة فوراً ونجح البناء.

4. عدم تطابق Enum اسم الدور في الـ Authorization Handler

وصف المشكلة: كتابة `MembershipRole.Manager` في حين أن الـ Domain معرف باسم `MembershipRole.PropertyManager`.

الحل: تصحيح الاسم ليكون `MembershipRole.PropertyManager`، وتحديث مصفوفة الصلاحيات بالكامل لتتوافق معه.

5. سقوط اختبار الـ Unit Test عند مسح الفحص المزدوج من الـ Handler

وصف المشكلة: عندما قمنا بتعديل الـ Handlers للاعتماد على الـ Repository، قمنا بحذف سطر `if (property.OrganizationId ≠ request.OrganizationId)` من داخل الـ Handler اعتمادًا على أن الـ Repository سيرجع null. لكن في الـ Unit Tests كان الـ Mock للـ Repository يرجع الـ Property، مما جعل الـ Test يفشل.

الحل المعماري: مبدأ **Defense-in-Depth** (الدفاع المتعدد الطبقات): جعل الـ Repository يفحص المنظمة، والـ Handler يفحص المنظمة، والـ DbContext يفحص المنظمة. بهذا ضمنّا أعلى درجات الحماية ونجاح جميع اختبارات الـ Unit Tests (45/45).

4. المقايضات الهندسية (Architectural Trade-offs)

القرار المعماري	المزايا (Pros)	العيوب والمقايضات (Cons & Trade-offs)	القرار المعتمد ولماذا؟
Global Query Filters مقابل Filtering يدوي في الـ Repositories	▪ حماية مركزية تلقائية ▪ استحالة نسيان سطر `Where` في أي استعلام جديد	▪ تتطلب الحذر الشديد لمنع الـ Fail-Open ▪ صعوبة التعامل مع العمليات النظامية المجمعة (Background Jobs)	**اعتمدنا الاثنين معاً (Defense-in-Depth):** الفلتر المركزي يحمي كل شيء افتراضياً، والـ Repository يمرر الـ OrgId صراحة كطبقة أمان إضافية.
Claims-Based OrgId مقابل Header `X-Organization-Id`	▪ موثقة ومحمية رقميًا ضد التلاعب داخل الـ JWT ▪ لا يمكن للعميل تزييفها	▪ في حالة كان المستخدم ينتمي لأكثر من شركة، يحتاج لعمل Switch Tenant وتوليد Token جديد	**Claims-Based:** لأن الحماية والأمان هي الأولوية القصوى لمنع أي محاولة انتحال هوية بين الشركات.
Fine-Grained Permissions مقابل الأدوار الثابتة (Simple RBAC)	▪ مرونة فائقة؛ إمكانية تعديل صلاحيات أي دور مستقبلاً من قاعدة البيانات ▪ الكود في الـ Controllers يعتمد على "ما الذي يسمح بفعله" وليس "من أنت"	▪ استهلاك طفيف في الذاكرة لفحص الـ HashSet في الـ Authorization Handler	**Fine-Grained (PBAC):** لأن الـ SaaS يتطور، وغدًا قد يحتاج العميل تخصيص أدوار جديدة (Custom Roles) بسهولة.
Optimistic Concurrency (RowVersion) مقابل Pessimistic DB Locking	▪ أداء وسرعة فائقة بدون حجز Locks في SQL Server ▪ مقياس عالٍ لقابلية التوسع (Scalability)	▪ يتطلب من واجهة المستخدم إعادة المحاولة (Retry) في حالة حدوث تضارب في التعديل المتزامن	**Optimistic Concurrency:** معيار ذهبي لتطبيقات الويب والـ Cloud الحديثة.

5. نتائج التحقق والاختبارات (Verification & CI Metrics)

تم تشغيل جميع مشاريع الاختبارات على منصة GitHub Actions باستخدام **.NET 10**، وكانت النتائج خضراء بنسبة 100%:

- 44 **PropertyManagement.Domain.Tests:** اختبار ناجح (0 فشل).
- 45 **PropertyManagement.Application.Tests:** اختبار ناجح (0 فشل).
- 3 **PropertyManagement.IntegrationTests:** اختبارات عزل مستأجرين شاملة ناجحة (0 فشل).
- **إجمالي الاختبارات: 92 اختبار ناجح بنسبة نجاح 100%.**
- **Verify Docker Container Build:** نجاح تام في بناء وتشغيل حاوية الـ Docker للـ API.

6. الخاتمة والخطوة القادمة (Next Milestone)

بهذا الإنجاز، أصبحت نواة الـ Multi-Tenancy ونظام الـ Authorization محصنة ضد كافة ثغرات الـ IDOR والـ Cross-Tenant Leaks ومطابقة لأعلى معايير الـ Production Grade.

الخطوة التالية المباشرة هي **Milestone 6: Tenant Invitations & Team Management** لإتاحة دعوة أعضاء الفريق، وإرسال Tokens مشفرة، وتفعيل العضويات بالأدوار المناسبة.